

SMARTMOVE TRANSPORT SOLUTIONS

FULL COURSEWORK PROJECT GUIDE

Data Management 2 - CW No 1

Complete requirement breakdown, project architecture, member responsibilities, implementation roadmap, testing, demo and submission readiness

Item	Assessment-sheet information
Institution	National Institute of Business Management - School of Computing and Engineering
Course	HDSE HNDIS
Module	Data Management 2
Assessment	CW No 1
Mode	Group
Maximum group size	4 members
Assessment type	Software Presentation VIVA
Submission detail	GitHub link of the project
Stated hand-in date	31.08.2026
Stated hand-out date	28.09.2026

SOURCE STATUS

The first part of this guide records what the uploaded NIBM assessment sheet actually requires. Later sections add clearly labelled recommended engineering choices and planning guidance; those recommendations are not additional lecturer-mandated requirements.

IMPORTANT ASSESSMENT POLICY

The assessment sheet states: "AI generated contents will obtain 0 marks." Use this guide for planning and understanding, but ensure the submitted implementation and presentation comply with your lecturer's academic-integrity rules and that the team can explain its own work.

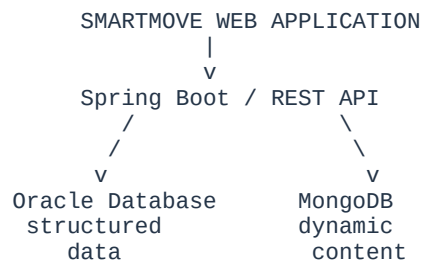
1. What This Coursework Actually Is

The SmartMove CW is not just a database exercise and it is not just a website. It is a hybrid database application project that combines a web/enterprise application, an Oracle relational database, PL/SQL business logic and reports, MongoDB for dynamic or unstructured transportation information, and a live presentation/viva.

The project scenario describes SmartMove Transport Solutions as a transportation service provider handling buses, vans and other vehicles for daily commuters, corporate employees and special travel requirements. The system manages vehicle operations, driver assignments, passenger registration, route planning, trip scheduling, ticket reservations, payment processing and customer feedback.

THE SIMPLE IDEA

Think of the final project as one transport-management product with two databases behind it. Oracle stores the structured transactional system of record; MongoDB complements it with flexible content such as reviews, documents, announcements and multimedia. The web application sits above both.



1.1 Learning outcomes stated on the sheet

- Identify the fundamentals of Oracle Database Server / SQL Server.
- Explain PL/SQL programming techniques.
- Apply Oracle for database administration.
- Use non-relational databases.

1.2 Assessment mode and practical implication

The assessment is a group assessment with a maximum of four members. Its stated assessment type is Software | Presentation | VIVA. Therefore the team must not only submit source code; it must also be prepared to demonstrate the software and explain the technical decisions in a viva.

2. Exact CW Requirement Breakdown

The following list is the project requirements taken from the uploaded assessment sheet, reorganized for planning clarity. It should be treated as the master checklist.

ID	Requirement
R01	Design the ER or EER diagram.

ID	Requirement
R02	Develop a simple Web or Enterprise Application for SmartMove Transport Solutions using any suitable programming language or framework.
R03	Design and implement a relational database schema using Oracle Database.
R04	Support Vehicle Management.
R05	Support Driver Management.
R06	Support Route Management.
R07	Support Passenger Management.
R08	Support Trip Scheduling Management.
R09	Support Ticket Booking Management.
R10	Support Payment Processing.
R11	Support Maintenance Management.
R12	Support Feedback and Review Management.
R13	Populate the database with sufficient sample data to simulate a realistic application scenario.
R14	Implement PL/SQL programs such as procedures, functions, cursors and/or triggers with exception handling for database operations and business logic.
R15	Include at least five business reports in the PL/SQL implementation.
R16	Implement MongoDB as a complementary NoSQL database for unstructured and dynamic transportation-related information.
R17	MongoDB collections should capture vehicle images and documents.
R18	MongoDB collections should capture passenger reviews and feedback comments.
R19	MongoDB collections should capture travel announcements and notifications.
R20	MongoDB collections should capture trip-related multimedia content.
R21	Use a schema-flexible MongoDB format for dynamic content and community interactions.
R22	Demonstrate retrieving passenger reviews for a specific route.
R23	Demonstrate identifying highest-rated vehicles or drivers based on customer feedback.
R24	Demonstrate keyword search over customer

ID	Requirement
	complaints/feedback.
R25	Demonstrate retrieval of vehicle documents and multimedia information.
R26	Integrate MongoDB with the application layer for real-time content delivery and the interactive use described in the sheet.
R27	Prepare the stated deliverables: Presentation and Software Application with Database Implementation.

3. Submission, Dates, Deliverables and Important Notes

3.1 Submission format

The sheet states the submission detail as: GitHub link of the project.

3.2 Stated deliverables

- Presentation
- Software Application with Database Implementation

3.3 Date inconsistency in the supplied sheet

The sheet displays Hand in Date / Time as 31.08.2026 and Hand out Date / Time as 28.09.2026. Because a hand-in date appears earlier than the hand-out date, the dates are internally inconsistent. Do not silently assume a corrected deadline from this guide; confirm the final date with the lecturer or official NIBM communication.

3.4 Separate report requirement?

The supplied assessment sheet does not explicitly list a separate printed report or a separate per-member report as a deliverable. It explicitly lists the presentation and the software application with database implementation, with a GitHub link as the submission detail. Team documentation is still recommended for development and presentation, but that documentation should not be described as a mandatory deliverable unless the lecturer separately requires it.

3.5 Five business reports - what that means

"At least five business reports" refers to business-information outputs implemented through the PL/SQL/reporting part of the project. It does not mean five separate PDF/Word submission documents and it does not mean five reports per student. The five reports are best exposed in a Reports section of the application and backed by Oracle/PLSQL logic.

3.6 Academic-integrity warning

FOLLOW THE SHEET

The assessment sheet explicitly states that AI generated contents will obtain 0 marks. The safest project workflow is to use external assistance only in ways permitted by your lecturer, and to ensure the final team-created code, explanations and presentation are understood and defensible by the group.

4. Marking Rubric and How to Build for the Highest Band

Criterion	Weight	What the 70%+ column expects
Front-end Development + ER Diagram	20%	Fully functional, user-friendly front-end; ER diagram complete, normalized, and clearly represents all entities.
Database Implementation (Oracle)	15%	All operations implemented with PL/SQL, correct schema, strong security and backup mechanisms.
MongoDB Incorporation	15%	Correct MongoDB incorporation, requirements fully captured with standard queries.
Reports & Business Logic (PL/SQL)	15%	Comprehensive reports, robust PL/SQL logic, advanced exception handling.
Completeness of the Project	5%	Fully complete; all requirements achieved.
Presentation & Viva	20%	Excellent understanding, confident communication, demonstrates deep knowledge.
Integration & Innovation	10%	Strong integration with Oracle + MongoDB and innovative/hybrid use of both systems.

PRIORITY

The largest single technical danger is not a missing CSS effect. It is an incomplete requirement chain: a feature that exists in the UI but is not truly backed by the database, a PL/SQL function that is never used, MongoDB that is never integrated into the app, or a report that cannot be demonstrated.

4.1 Rubric-driven development principle

Requirement -> Design -> Database -> PL/SQL/API -> UI -> Test -> Evidence -> Demo

Do not build disconnected pieces and try to join them at the end.

4.2 Rubric anomaly

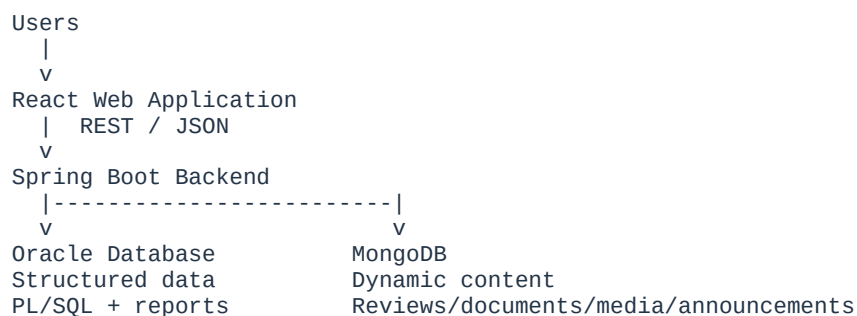
The supplied rubric visibly lists 0-39%, 40-49%, 50-59% and 70%+ columns but does not show a 60-69% band. Treat that as a presentation/design anomaly in the supplied sheet rather than inventing a missing band.

5. Recommended Full Project Architecture

The assessment allows any suitable language/framework for the Web or Enterprise Application. The following stack is therefore a recommendation, not a lecturer-mandated stack.

Layer	Recommended choice	Purpose
Frontend	React + TypeScript + Vite + Tailwind CSS	User-facing web application
UI components	Reusable component library such as shadcn/ui	Consistent forms, tables, dialogs and cards
Charts	Recharts or equivalent	Report visualization
Backend	Spring Boot + Java 17 + Maven	REST API, validation, security and integration
Relational database	Oracle Database	Structured transactional data and PL/SQL
NoSQL database	MongoDB	Dynamic/unstructured transportation content
Oracle access	Spring Data JPA / JDBC / stored procedure support	CRUD and PL/SQL integration
Mongo access	Spring Data MongoDB	MongoDB application-layer access
API documentation	OpenAPI/Swagger	Makes backend interfaces easy to test and explain
API testing	Postman or equivalent	Independent backend verification
Oracle tooling	Oracle SQL Developer or equivalent	Schema/PLSQL administration and testing
Mongo tooling	MongoDB Compass or shell	Collection/query verification
Version control	Git + GitHub	Collaboration and required project submission

5.1 Architecture diagram



5.2 Why two databases?

Oracle is the natural home for structured entities and relationships that drive bookings, payments, trips and operational records. MongoDB is intended by the assignment to complement Oracle with dynamic/unstructured transportation information such as reviews, documents, notifications and multimedia. The project should demonstrate a deliberate split of responsibilities rather than duplicate the same data in both systems.

6. Proposed Business Domain and Relational Model

The assessment names nine core system operations. A practical relational model therefore centers on the following core entities. The exact attributes and relationship cardinalities must be finalized by Member 1 and agreed by the whole group before implementation.

Core entity	Main purpose
VEHICLE	Stores vehicle identity, type, capacity and operational status.
DRIVER	Stores driver identity and assignment-related information.
ROUTE	Defines an origin/destination route and any agreed route metadata.
PASSENGER	Stores registered passenger information.
TRIP	Represents a scheduled service using a route, vehicle and driver.
BOOKING	Represents a passenger reservation for a trip.
PAYMENT	Stores payment information associated with a booking.
MAINTENANCE	Records maintenance activity and due/next-maintenance information.
FEEDBACK	Stores structured feedback where the relational design uses it.

6.1 High-level relationships

```
PASSENGER 1 ---- * BOOKING * ---- 1 TRIP
TRIP * ---- 1 ROUTE
TRIP * ---- 1 VEHICLE
TRIP * ---- 1 DRIVER
BOOKING 1 ---- * PAYMENT (or the exact cardinality agreed by the team)
VEHICLE 1 ---- * MAINTENANCE
PASSENGER / TRIP / VEHICLE / DRIVER ---- FEEDBACK as defined by the final ERD
```

DESIGN DISCIPLINE

Do not lock the exact cardinality of every relationship from this recommended sketch. Member 1 must produce the final ER/EER based on the actual business rules; Member 3 then implements that final schema.

7. The Nine Required Core Operations

Operation	What the finished project should demonstrate
1. Vehicle Management	Create, view, update and manage vehicles; expose useful status and capacity data; connect maintenance and MongoDB vehicle content where relevant.
2. Driver Management	Create, view, update and manage drivers; support assignment to trips and any agreed performance/rating view.
3. Route Management	Create and maintain routes; make routes searchable and usable by trips and reporting.
4. Passenger Management	Register and manage passengers; provide passenger data needed for bookings and travel history.
5. Trip Scheduling Management	Create scheduled trips linking routes, vehicles and drivers, with appropriate availability/status rules.
6. Ticket Booking Management	Allow passengers to reserve trips; enforce seat/availability and booking business rules.
7. Payment Processing	Record a payment process linked to the booking; use a simulated academic payment flow unless a real integration is separately required.
8. Maintenance Management	Record and monitor vehicle maintenance; identify vehicles due for maintenance.
9. Feedback and Review Management	Capture customer feedback and expose review information; complement structured records with MongoDB dynamic review content where appropriate.

8. The Five Business Reports

The sheet provides four examples. The fifth report below is a team recommendation so the required minimum is met with a useful analytics scenario.

#	Business report	Recommended output
Report 1	Most Frequently Used Routes	Rank routes by ticket-booking activity; show booking count and meaningful ranking.
Report 2	Total Revenue Within a Period	Accept a date range and calculate ticket-sale revenue for that period.
Report 3	Passenger Travel History	Show a passenger's historical trips/bookings, route, vehicle and related payment status where available.

#	Business report	Recommended output
Report 4	Vehicles Due for Maintenance	Find vehicles whose maintenance is due based on the project's maintenance date/business rule.
Report 5 - recommended	Trip Occupancy & Performance	Show capacity, booked seats, available seats, occupancy percentage and revenue for trips.

8.1 Recommended report pipeline

```

React Reports Page
  |
  v
Spring Boot /reports API
  |
  v
Oracle PL/SQL / SQL report logic
  |
  v
JSON result -> table/chart in React

```

9. MongoDB Requirements - Exact Coverage

MongoDB is explicitly required as a complementary NoSQL database. The sheet asks for schema flexibility and four content categories, plus four query demonstrations and application-layer integration.

Collection area	What it should support	Visible application use
vehicle_documents	Vehicle images, document metadata and related dynamic fields.	Vehicle detail page with images/documents.
reviews	Passenger reviews, ratings and feedback comments.	Route review list, top-rated view, keyword search.
announcements	Travel announcements and notifications.	Announcement/notification area.
trip_media	Trip-related multimedia metadata/content references.	Trip media gallery or trip detail content.

9.1 Required MongoDB queries

1. Retrieve all passenger reviews for a specific route.
2. Identify the highest-rated vehicles or drivers based on customer feedback.
3. Search customer complaints or feedback using specific keywords.
4. Retrieve vehicle documents and multimedia information.

9.2 Schema flexibility - what it should mean in this project

Schema-flexible means the MongoDB document model can naturally accommodate optional/dynamic content without forcing every record to have an identical rigid relational shape. It does not mean the

database should be chaotic. Keep common identifiers and useful standard fields while allowing optional metadata where it adds value.

9.3 Oracle/MongoDB linkage

Use stable IDs to connect dynamic MongoDB content to relational entities. For example, a MongoDB review can carry routeId, vehicleId, driverId, passengerId and tripId values that refer back to Oracle entities, without copying complete relational records into MongoDB.

10. Full 4-Member Responsibility Structure

The chosen team model is 2 members focused on theory/design/documentation and 2 members focused on coding/technology. Ownership is clear, but integration is shared: nobody can claim the whole project is somebody else's responsibility.

Member	Primary role	Key ownership
Member 1	System Analyst + ERD + Database Designer	Requirements, ER/EER, normalization, data dictionary, Oracle schema design, design QA, business process model.
Member 2	PL/SQL + Reports + Documentation/Viva Analyst	PL/SQL specifications, five report specifications, MongoDB design/specs, test plan, presentation, viva material, evidence coordination.
Member 3	Oracle + PL/SQL + Spring Boot Developer	Oracle implementation, PL/SQL implementation, reports, REST API, backend, authentication, Oracle integration, technical integration.
Member 4	React + MongoDB + UX Developer	Frontend, MongoDB collections/sample data/queries, Mongo-backed UI, UX, responsive design, frontend testing, integration UI.

IMPORTANT

Theory members should not become "document-only" people. Members 1 and 2 need enough technical understanding to defend the project in the viva, while Members 3 and 4 need enough theoretical understanding to explain why the system is designed the way it is.

11. MEMBER 1 - Complete Responsibility Package

Member 1 is the master system-design and relational-data architect. Their job is to decide what the system means and how its structured data is organized before the technical implementation is locked.

Requirements and analysis

- Break the CW into functional requirements, entities, workflows, database needs, PL/SQL needs, MongoDB needs, report needs and integration needs.
- Create a requirement traceability matrix.
- Identify any ambiguity and flag it to the team rather than silently inventing a rule.

ER/EER

- Produce the final ER/EER diagram.
- Define entities, attributes, keys, relationships, cardinalities and optionality.
- Keep the diagram synchronized with the actual Oracle schema.

Normalization

- Explain 1NF, 2NF and 3NF for the project as applicable.
- Identify repeating groups, partial dependencies and transitive dependencies where relevant.
- Prepare clear viva explanations for why tables are separated.

Data dictionary

- For each table/attribute define name, purpose, type, key role and constraints.
- Make the dictionary detailed enough for Member 3 to implement without guessing.

Oracle design

- Define the final table structure, PK/FK strategy, constraints, indexes where needed and any agreed views.
- Review schema changes proposed during implementation.

Design QA

- Compare the final Oracle schema against the ERD.
- Check that all nine required operations are represented in the data model.
- Check that report data can actually be produced from the schema.

Security/backup design support

- Define what security and backup mechanisms the team intends to demonstrate so Member 3 can implement them.
- Keep claims realistic and tied to what the team can actually show.

Member 1 handover to technical team

```
01_requirements.md
02_final_erd.png / pdf
03_normalization_notes.md
04_data_dictionary.xlsx / md
05_oracle_schema_spec.md
06_business_processes.md
```

12. MEMBER 2 - Complete Responsibility Package

Member 2 is the technical-planning, reports, testing and communication owner on the theory/documentation side. Their work turns the database/system design into a precise implementation checklist and strong evidence package.

PL/SQL specification

- Specify the required procedures, functions, cursors, triggers and exception cases.
- For each program document purpose, input, output, main validation/business rule and expected failure cases.

Five report specifications

- Define purpose, filters, required columns, calculation, expected output and business meaning for each report.
- Ensure the fifth report is formally chosen and agreed by the team.

MongoDB design/specification

- Define the four required collection areas.
- Define document examples, field expectations, identifier links and schema-flexible elements.
- Translate the four required query scenarios into implementation/test cases.

Testing plan

- Create positive, negative and integration test cases.
- Define expected results and evidence requirements.
- Track failures until they are fixed.

Presentation

- Own the structure and evidence flow of the final presentation.
- Collect screenshots and concise explanations from all four members.
- Make sure the presentation mirrors the actual implementation.

Viva preparation

- Build a group question bank covering Oracle, PL/SQL, MongoDB, application architecture, integration, business rules and reports.
- Schedule team rehearsal so each member can answer beyond their individual module.

Evidence coordination

- Maintain a list of screenshots, query results, test results and live-demo checkpoints.
- Ensure evidence proves actual functionality rather than only showing source code.

Member 2 handoff to technical team

```
plsql_specification.md
five_report_specifications.md
mongodb_specification.md
mongodb_query_cases.md
test_cases.xlsx / md
presentation_outline.md
viva_question_bank.md
```

13. MEMBER 3 - Complete Responsibility Package

Member 3 is the core technical engine owner: Oracle implementation, PL/SQL, reports, Spring Boot backend, REST APIs, security and integration support.

Oracle implementation

- Create tables from the final ERD/data dictionary.
- Implement PK/FK/UNIQUE/NOT NULL/CHECK constraints as designed.
- Implement sequences or the agreed ID strategy.
- Create indexes where justified.
- Populate realistic sample data.
- Create repeatable SQL setup scripts.

PL/SQL implementation

- Implement meaningful procedures for business operations such as registration, trip scheduling, booking, payment, maintenance and cancellation where agreed.
- Implement useful functions such as available seats, revenue, occupancy and rating calculations where agreed.
- Implement cursors where they add genuine report/processing value.
- Implement triggers for automatic business-state changes where appropriate.
- Implement robust exception handling and safe transaction logic.

Five reports

- Implement all five reports in Oracle/PLSQL.
- Test reports with non-trivial sample data.
- Expose report results through backend endpoints so the UI can demonstrate them.

Spring Boot backend

- Create a clean Spring Boot project.
- Implement REST controllers, services, repositories/PLSQL access and DTOs as appropriate.
- Add backend validation and consistent error handling.
- Document API contracts for Member 4.

Authentication and authorization

- Implement login/role behavior appropriate to the project.
- Enforce authorization on the backend; do not rely only on frontend hiding.
- Keep credentials and secrets out of source control.

Oracle integration

- Connect application services to Oracle.
- Use PL/SQL for agreed business-critical operations rather than leaving the PL/SQL layer isolated.
- Make report endpoints call the real report logic.

MongoDB backend integration

- Connect Spring Boot to MongoDB using an appropriate driver/library.

- Expose review, announcement, document and media data to the application.
- Support the four required MongoDB scenarios through backend services/endpoints.

Testing and technical handover

- Run API/database/integration tests.
- Capture technical evidence.
- Provide API documentation, setup instructions and known limitations to Member 4 and Member 2.

14. MEMBER 4 - Complete Responsibility Package

Member 4 is the user-facing and MongoDB content-layer owner: React application, UI/UX, MongoDB collections and queries, and frontend integration with the Spring Boot APIs.

Frontend foundation

- Set up React/TypeScript/Vite and a consistent component system.
- Create the application shell, navigation, layouts, reusable forms, tables, modals, cards, badges and loading/error states.

Required UI modules

- Build vehicle, driver, route, passenger, trip, booking, payment, maintenance and feedback screens.
- Build the five-report UI.
- Build MongoDB-backed reviews, announcements, vehicle documents and trip-media views.

Booking and passenger journey

- Provide trip search and selection.
- Show availability supplied by the backend.
- Drive booking -> payment -> confirmation flow.
- Show travel history.

MongoDB implementation

- Create vehicle_documents, reviews, announcements and trip_media collections or the final agreed equivalents.
- Insert realistic sample documents.
- Implement and verify the four required query scenarios.

Mongo-backed UI

- Show route reviews.
- Show top-rated vehicles/drivers.
- Provide complaint/feedback keyword search.
- Show vehicle documents/images and trip media.
- Display announcements/notifications.

API integration

- Connect all UI pages to Member 3 APIs.

- Respect request/response formats.
- Handle authentication and role-dependent navigation.
- Translate backend errors into user-friendly messages.

UX and quality

- Responsive layout.
- Consistent typography, spacing and controls.
- Clear empty states, validation and loading indicators.
- Avoid hard-coded business results.

Frontend testing and evidence

- Run full user journeys.
- Capture screenshots of important screens and MongoDB-backed features.
- Provide integration evidence to Member 2.

15. Full Team Development Workflow

The team should work sequentially where design dependencies matter, then iterate in parallel. The following workflow reduces rework and keeps the project traceable.

Phase	Lead	Work
Phase 1 - Requirement Freeze	All four members	Turn the CW into a master checklist. Agree scope, roles, core operations, report list, MongoDB collections and technology choices.
Phase 2 - Data and system design	M1 + M2 lead	Finalize ERD, normalization, data dictionary, report specifications, PL/SQL specification, MongoDB design, API needs and test plan.
Phase 3 - Technical skeleton	M3 + M4	Create Spring Boot and React projects, Git branches, database connections, shared naming conventions and development environments.
Phase 4 - Oracle foundation	M3 lead, M1 review	Implement schema, constraints, IDs, sample data, security/backup components and test basic queries.
Phase 5 - PL/SQL and reports	M3 lead, M2 review	Implement procedures, functions, cursors, triggers, exceptions and five reports.
Phase 6 - Backend APIs	M3 lead	Implement REST endpoints, authentication, validation, report endpoints and MongoDB-backed services.
Phase 7 - Frontend + MongoDB	M4 lead	Build all required screens, MongoDB collections/data/queries and connect to APIs.
Phase 8 - Integration	M3 + M4	Run Oracle-backed and MongoDB-backed workflows from the browser. Fix contract and data-flow problems.

Phase	Lead	Work
Phase 9 - QA and evidence	M2 coordinates, all members test	Run positive/negative/end-to-end tests. Capture evidence. Recheck the requirement matrix.
Phase 10 - Presentation/viva	All four	Rehearse live demo, explain design decisions, verify GitHub and freeze the release version.

16. Recommended GitHub Repository Structure

```

SmartMove-Transport-Solutions/
|-- frontend/
|   |-- smartmove-react/
|-- backend/
|   |-- smartmove-springboot/
|-- database/
|   |-- oracle/
|   |   |-- tables/
|   |   |-- sequences/
|   |   |-- constraints/
|   |   |-- indexes/
|   |   |-- views/
|   |   |-- procedures/
|   |   |-- functions/
|   |   |-- cursors/
|   |   |-- triggers/
|   |   |-- reports/
|   |   |-- security/
|   |   |-- backup/
|   |   |-- sample-data/
|   |-- mongodb/
|   |   |-- collections/
|   |   |-- sample-data/
|   |   |-- queries/
|-- docs/
|   |-- ERD/
|   |-- architecture/
|   |-- api/
|   |-- testing/
|   |-- presentation/
|-- testing/
|-- README.md
|-- .gitignore

```

16.1 Git rules

- Use feature/role branches and merge through a stable development branch.
- Commit logical changes with meaningful messages.
- Never commit database passwords, API secrets, JWT secrets or private credentials.
- Do not use the repository as a dumping ground for huge temporary files.
- Tag or otherwise identify the final demo/release version.

17. Oracle Implementation Checklist - Technical Detail

17.1 Foundation objects

- Tables for all required structured business entities.
- Primary keys.
- Foreign keys.
- NOT NULL constraints.
- UNIQUE constraints where appropriate.
- CHECK constraints where appropriate.
- ID-generation strategy (sequences or supported identity approach).
- Useful indexes where justified.
- Optional views that materially simplify repeated reporting queries.

17.2 Sample data design

The dataset should be large enough that reports produce meaningful differences. A practical recommendation is to have repeated routes, multiple trips, multiple passengers, multiple bookings and payments, maintenance records in different states, and several feedback records. The exact quantity is not prescribed by the sheet; the requirement is "sufficient sample data".

17.3 Security and backup

The rubric's highest Oracle band explicitly mentions strong security and backup mechanisms. The team should implement a demonstrable, realistic security/backup plan that matches the lab environment rather than claiming enterprise capabilities it cannot show.

18. PL/SQL Implementation Plan - Technical Detail

18.1 Procedures - recommended business operations

- Register vehicle
- Register passenger
- Schedule trip
- Assign driver
- Create booking
- Process payment
- Cancel booking
- Record maintenance

18.2 Functions - recommended calculation/lookup operations

- Get available seats
- Calculate trip revenue
- Calculate route/trip occupancy
- Calculate driver rating, if ratings are modeled

18.3 Cursors

Use cursors where iterating through a result set adds clear report/processing value. Be able to explain OPEN, FETCH, EXIT and CLOSE if an explicit cursor is used, or the equivalent cursor-for-loop behavior.

18.4 Triggers

- Automatically maintain related status after a valid business event where appropriate.
- Keep booking/payment or maintenance-related states consistent if triggers are part of the final design.
- Avoid unnecessary trigger chains and hidden side effects.

18.5 Exception handling

- Missing passenger
- Missing trip
- Unavailable vehicle
- Unavailable driver
- No seats available
- Missing booking
- Duplicate data
- Invalid amount/date or other domain-specific rule

DO NOT

Do not hide all errors with WHEN OTHERS THEN NULL. The goal is robust, understandable exception handling and safe database state.

18.6 Transactions

Booking and payment flows should be designed with clear transaction boundaries. If a critical step fails, the operation should not leave partial or contradictory data. Test both success and failure cases.

19. Spring Boot Backend - What Must Exist

Layer	Suggested responsibility
Controller	Expose REST endpoints, validate request shape, map responses.
Service	Business orchestration, transaction boundaries, calls to repositories/stored procedures.
Repository / data access	Oracle/JPA/JDBC access and MongoDB repository access.
DTO	Stable API request/response contracts instead of exposing every internal entity directly.
Security	Authentication, authorization and protected routes.
Exception handler	Map internal errors into clean API responses.
Reports module	Call report logic and return structured result data.

Layer	Suggested responsibility
Mongo module	Expose reviews, announcements, documents and media through the application layer.

19.1 Suggested API groups

/api/auth
 /api/vehicles
 /api/drivers
 /api/routes
 /api/passengers
 /api/trips
 /api/bookings
 /api/payments
 /api/maintenance
 /api/feedback
 /api/reports
 /api/reviews
 /api/announcements
 /api/documents
 /api/media

19.2 Important API principle

Member 4 should consume stable, documented API contracts. Any change to endpoint names, request fields or response fields should be communicated and tested before merging.

20. Frontend Application - Complete Screen Plan

Area	Recommended screens / functions
Authentication	Login, registration, logout, protected application access.
Dashboard	Vehicle count, drivers, trips, bookings, revenue, maintenance indicators and selected charts.
Vehicles	List, search/filter, add, edit, view details, status, maintenance context, Mongo documents/images/reviews.
Drivers	List, add, edit, view, assignment-related information and rating/performance where supported.
Routes	List, add, edit, view and use in trip/report workflows.
Passengers	Registration, list/detail, profile/travel-history access as appropriate.
Trips	List, schedule/create, details, route/vehicle/driver assignment.
Bookings	Search trips, book, booking details, cancellation where supported.
Payments	Review amount, simulated payment flow, success/failure state.
Maintenance	List, record maintenance, due view/report.
Feedback	Submit rating/comment and view relevant feedback.
Reports	All five required business reports with filters and tables/charts

Area	Recommended screens / functions
	where useful.
Mongo content	Reviews, announcements, vehicle documents/images, trip media.

20.1 UX minimum quality

- Every API-driven page handles Loading / Success / Empty / Error states.
- Forms have clear labels and useful validation messages.
- Tables remain usable at common laptop widths.
- Navigation is consistent and role-aware.
- Important actions have confirmation feedback.
- Charts and dashboard numbers come from real data.

21. Oracle + MongoDB Integration - The Key Hybrid Demonstration

The integration criterion is worth 10% and the top rubric band calls for strong Oracle + MongoDB integration. The team should therefore demonstrate a single user experience that combines data from both databases.

21.1 Strong hybrid example - vehicle detail

Vehicle Detail: Vehicle #25

Oracle:

Registration | Type | Capacity | Status | Driver | Maintenance

MongoDB:

Images | Documents | Reviews | Dynamic media

Spring Boot combines the two sources for the React page.

21.2 Strong hybrid example - customer review flow

```

Passenger books trip
  |
  v
Trip/Passenger/Booking data in Oracle
  |
  v
Passenger submits dynamic review content
  |
  v
Review stored in MongoDB
  |
  v
Review retrieved by route / vehicle / driver queries
  |
  v
React displays the results

```

WHY THIS MATTERS

The lecturer should be able to see that MongoDB is not a disconnected database exercise. It is part of the working application and complements Oracle in a deliberate way.

22. Full Testing Strategy

22.1 Functional testing

- Login success/failure
- Vehicle CRUD
- Driver CRUD
- Route CRUD
- Passenger management
- Trip scheduling
- Booking success
- Booking failure when rules are violated
- Payment success/failure
- Maintenance records
- Feedback submission
- All five reports
- All four MongoDB scenarios

22.2 Database/PLSQL testing

- Constraints reject invalid data.
- Foreign keys protect relationships.
- Procedures succeed with valid input.
- Procedures fail cleanly with invalid input.
- Functions return correct results using known sample data.
- Triggers fire only when intended.
- Exceptions are meaningful.
- Transactions do not leave partial data on critical failure.

22.3 Integration testing

- React -> Spring Boot -> Oracle.
- React -> Spring Boot -> PL/SQL.
- React -> Spring Boot -> MongoDB.
- Oracle data and MongoDB content can appear together in at least one meaningful screen.
- Report data displayed in React matches database output.

22.4 Suggested end-to-end acceptance scenario

1. Login as passenger
2. Search trips
3. Select trip
4. Create booking
5. Process simulated payment

6. View booking confirmation
7. Submit a review
8. Verify review is visible through Mongo-backed UI
9. Login as admin
10. View vehicle/driver/route/trip data
11. Record maintenance
12. Run all five reports
13. Demonstrate four required MongoDB queries
14. Show an Oracle + MongoDB hybrid page

23. Evidence and Demonstration Checklist

Area	Evidence to prepare
ERD	Final ER/EER image/PDF and explanation of main relationships.
Oracle	Schema objects, sample data and representative table/query results.
PL/SQL	Procedure/function/cursor/trigger execution evidence and exception examples.
Reports	Result for all five reports using realistic sample data.
MongoDB	Collections, sample documents and all four required query results.
Frontend	Dashboard, core management pages, booking flow, reports and Mongo content screens.
Integration	A page/flow showing Oracle and MongoDB working together through the application.
Testing	Positive and negative test results, including important edge cases.
Security/backup	Evidence or documentation of what the team can actually demonstrate.
GitHub	Clean repository, README, setup instructions and reproducible project structure.

24. Presentation and Viva Plan

Presentation and viva account for 20% of the marks. Every member should therefore understand the whole project, not only their assigned files.

24.1 Suggested presentation flow

5. 1. Title and team
6. 2. SmartMove problem/scenario
7. 3. Objectives and requirements
8. 4. System architecture
9. 5. ER/EER diagram
10. 6. Oracle database design/implementation
11. 7. PL/SQL and business logic

12. 8. Five business reports
13. 9. MongoDB model and required queries
14. 10. Frontend and key workflows
15. 11. Oracle + MongoDB integration
16. 12. Security/backup/testing
17. 13. Innovation/hybrid analytics
18. 14. Live demo
19. 15. Conclusion

24.2 Viva areas every member must know

- Why Oracle for structured data?
- Why MongoDB for dynamic/unstructured content?
- Why the ERD is structured this way?
- Why normalization matters?
- What is a procedure? Function? Cursor? Trigger?
- How does exception handling protect the application?
- How does booking work end-to-end?
- How is revenue calculated?
- How does a report reach the UI?
- How does MongoDB reach the UI?
- Why should the browser not connect directly to Oracle/MongoDB?
- How is authentication/authorization handled?
- What would happen if a booking fails halfway?

25. Master Requirement-to-Owner Matrix

Area	M1	M2	M3	M4
ER/EER diagram	M1	M2 review	All understand	All understand
Requirements/design	M1	M2	Review	Review
Oracle schema design	M1	Support	Implement	Consume
Oracle implementation	Review	Evidence	M3	Consume
Vehicle management	Design	Test spec	Backend/DB	UI
Driver management	Design	Test spec	Backend/DB	UI
Route management	Design	Test spec	Backend/DB	UI
Passenger management	Design	Test spec	Backend/DB	UI
Trip scheduling	Design	Business-rule spec	Backend/PLSQL	UI
Ticket booking	Design	Business-rule spec	Backend/PLSQL	UI
Payment	Design	Business-rule spec	Backend/Oracle	UI
Maintenance	Design	Report spec	Backend/Oracle	UI
Feedback/review	Design	Mongo/logic spec	Backend	UI + Mongo
PL/SQL	Spec review	Spec owner	Implementation	Consume via UI

Area	M1	M2	M3	M4
procedures/functions/cursors/triggers				
Exception handling	Review	Scenario/spec	Implementation	Display errors
Five reports	Spec review	Report owner	Oracle implementation/API	Report UI
MongoDB collections	Design support	Mongo spec owner	Backend integration	Implementation owner
MongoDB required queries	Spec/Test	Spec owner	Endpoint support	Implementation/testing
Frontend	ERD alignment	Evidence/demo	API provider	Implementation owner
Integration	Architecture review	Evidence	Technical lead	Technical UI lead
Testing	Design validation	Coordinates	Backend/database tests	Frontend/Mongo tests
Presentation/Viva	ERD section	Coordinates	Technical section	UI/Mongo section

26. Final Pre-Submission Checklist

Assessment basics

- ☐ Group has ≤ 4 members
- ☐ Final submission is the intended GitHub project link
- ☐ Team has confirmed the real deadline because the supplied sheet dates are inconsistent
- ☐ Team understands the AI-policy statement

Design

- ☐ Final ER/EER complete
- ☐ ERD matches actual schema
- ☐ Normalization justified
- ☐ Data dictionary complete

Oracle

- ☐ All nine operations represented
- ☐ Constraints work
- ☐ Realistic sample data
- ☐ Security mechanisms ready to demonstrate
- ☐ Backup approach ready to demonstrate/document

PL/SQL

- ☐ Procedures
- ☐ Functions
- ☐ Cursors where used
- ☐ Triggers where used
- ☐ Robust exception handling

- ☐ Transaction behavior tested

Reports

- ☐ Frequently used routes
- ☐ Revenue by period
- ☐ Passenger travel history
- ☐ Vehicles due maintenance
- ☐ Fifth report implemented and tested

MongoDB

- ☐ Vehicle images/documents
- ☐ Reviews/feedback
- ☐ Announcements/notifications
- ☐ Trip multimedia
- ☐ Schema-flexible document examples

MongoDB queries

- ☐ Route reviews query
- ☐ Highest-rated vehicle/driver query
- ☐ Keyword complaint search
- ☐ Vehicle documents/media retrieval

Application

- ☐ Web application starts
- ☐ Login/roles work if implemented
- ☐ All nine operations accessible
- ☐ Booking/payment flow works
- ☐ Reports visible
- ☐ MongoDB content visible
- ☐ Good loading/error/empty states

Integration

- ☐ React -> backend -> Oracle
- ☐ React -> backend -> MongoDB
- ☐ At least one clear hybrid Oracle + MongoDB page/flow
- ☐ No fake hard-coded business values

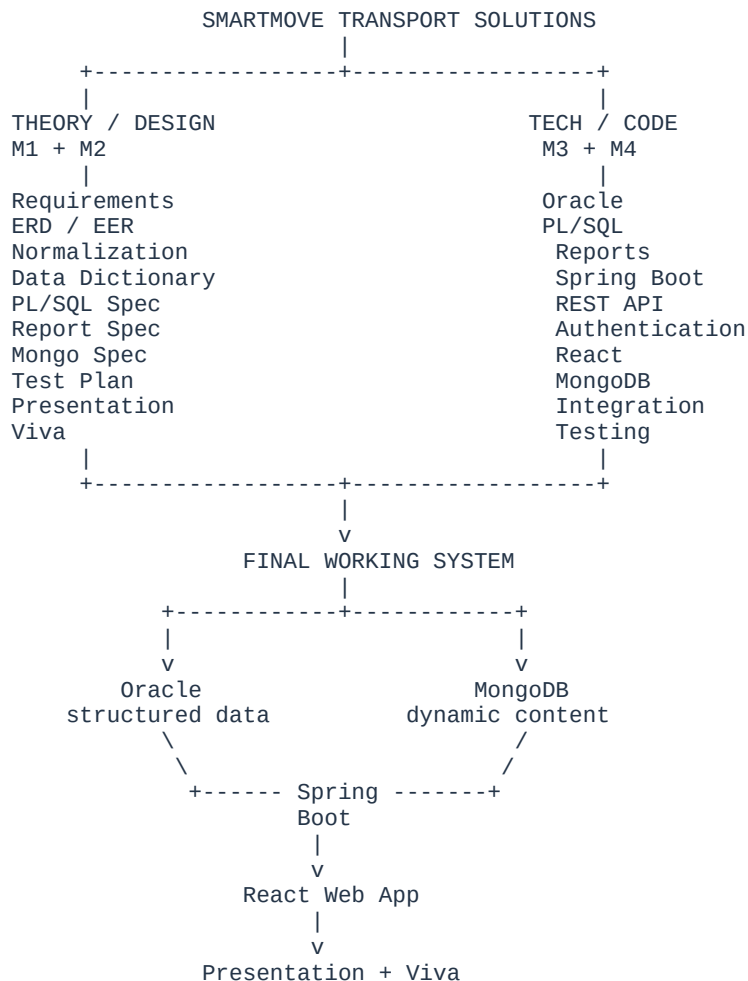
Testing and evidence

- ☐ Positive tests
- ☐ Negative tests
- ☐ End-to-end scenario
- ☐ Screenshots/evidence captured
- ☐ Presentation reflects the real implementation
- ☐ All members can explain the project

GitHub/readiness

- [] README setup guide
- [] Database scripts organized
- [] MongoDB queries stored
- [] No secrets committed
- [] Final release tested on a clean environment where practical

27. One-Page Project Map



THE WHOLE PROJECT IN ONE SENTENCE

Build one SmartMove web application whose structured transportation transactions and business logic are handled with Oracle + PL/SQL, whose dynamic/unstructured transportation content is handled with MongoDB, and whose frontend demonstrates the whole system end-to-end.

Source basis: uploaded NIBM School of Computing and Engineering, Data Management 2, CW No 1 Assessment Announcement Sheet (4 pages). This guide distinguishes source-derived requirements from recommended implementation/planning choices.